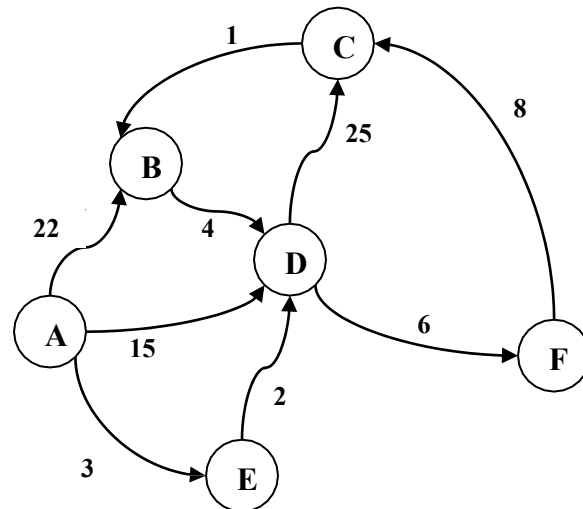
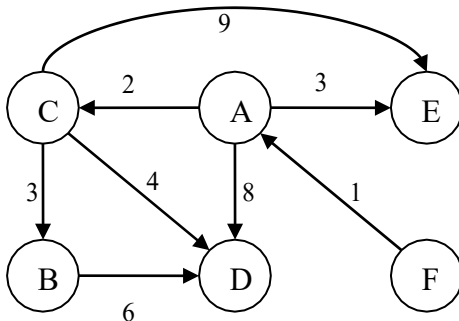


Lab Goal : This lab was designed to teach you how to solve a weighted shortest path problem using dijkstra's algorithm on an adjacency list.

Lab Description : The data file contains the Dijkstra's problems from the first Dijkstra's worksheet, so you can check your results against it. There are also two undirected graphs in the data file. All graphs are shown below. You will find all of the shortest paths from vertices given in the data file.

Sample Directed Graphs :



Sample Output :

(Your map output will be different if you used strings to store the neighbors.)

Graph #1 - Directed Graph

```
{A=[C2.0, D8.0, E3.0], B=[D6.0], C=[B3.0, D4.0, E9.0], D=[], E=[], F=[A1.0]}
```

Minimum Distances:

A	B	C	D	E	F
1.0	inf	inf	inf	inf	0.0
F	null	null	null	null	null

Minimum Distances:

A	B	C	D	E	F
1.0	inf	3.0	9.0	4.0	0.0
F	null	A	A	A	null

Minimum Distances:

A	B	C	D	E	F
1.0	6.0	3.0	7.0	4.0	0.0
F	C	A	C	A	null

Minimum Distances:

A	B	C	D	E	F
1.0	6.0	3.0	7.0	4.0	0.0
F	C	A	C	A	null

Minimum Distances:

A	B	C	D	E	F
1.0	6.0	3.0	7.0	4.0	0.0
F	C	A	C	A	null

algorithm help

Before the loop,

- make maps to store the costs and previous nodes and a set to keep track of the visited nodes.
- Map each vertex to a cost of Double.POSITIVE_INFINITY and previous as null.
- Set the cost of the starting node to zero.

Loop size times:

- Find the node with the smallest cost not yet visited and set is as the current node
- If there are no more reachable nodes, stop looping.
- Mark the current node as visited
- Loop thru all neighbors of the current node:
 - If the neighbor has already been visited:
 - Skip it
 - Calculate the tentative cost this neighbor
 - If the tentative cost is cheaper than the current cost stored for this neighbor:
 - Update the cheapest cost and prev

Minimum Distances:

A	B	C	D	E	F
1.0	6.0	3.0	7.0	4.0	0.0
F	C	A	C	A	null

Path from F to A: F -> A

Path from F to B: F -> A -> C -> B

Path from F to C: F -> A -> C

Path from F to D: F -> A -> C -> D

Path from F to E: F -> A -> E

Graphs 2 – 4 correspond to the three Dijkstra's problems on the back of the worksheet, so they're all using the second graph shown above.

Graph #2 - Directed Graph

{A=[B22.0, D15.0, E3.0], B=[D4.0], C=[B1.0], D=[F6.0, C25.0], E=[D2.0], F=[C8.0]}

Minimum Distances:

A	B	C	D	E	F
0.0	22.0	inf	15.0	3.0	inf
null	A	null	A	A	null

Minimum Distances:

A	B	C	D	E	F
0.0	22.0	inf	5.0	3.0	inf
null	A	null	E	A	null

Minimum Distances:

A	B	C	D	E	F
0.0	22.0	30.0	5.0	3.0	11.0
null	A	D	E	A	D

Minimum Distances:

A	B	C	D	E	F
0.0	22.0	19.0	5.0	3.0	11.0
null	A	F	E	A	D

Minimum Distances:

A	B	C	D	E	F
0.0	20.0	19.0	5.0	3.0	11.0
null	C	F	E	A	D

Minimum Distances:

A	B	C	D	E	F
0.0	20.0	19.0	5.0	3.0	11.0
null	C	F	E	A	D

Path from A to B: A -> E -> D -> F -> C -> B

Path from A to C: A -> E -> D -> F -> C

Path from A to D: A -> E -> D

Path from A to E: A -> E

Path from A to F: A -> E -> D -> F

Graph #3 - Directed Graph

{A=[B22.0, D15.0, E3.0], B=[D4.0], C=[B1.0], D=[F6.0, C25.0], E=[D2.0], F=[C8.0]}

Minimum Distances:

A	B	C	D	E	F
inf	0.0	inf	4.0	inf	inf
null	null	null	B	null	null

Minimum Distances:

A	B	C	D	E	F
inf	0.0	29.0	4.0	inf	10.0
null	null	D	B	null	D

Minimum Distances:

A	B	C	D	E	F
inf	0.0	18.0	4.0	inf	10.0
null	null	F	B	null	D

Minimum Distances:

A	B	C	D	E	F
inf	0.0	18.0	4.0	inf	10.0
null	null	F	B	null	D

Minimum Distances:

A	B	C	D	E	F
inf	0.0	18.0	4.0	inf	10.0
null	null	F	B	null	D

Minimum Distances:

A	B	C	D	E	F
inf	0.0	18.0	4.0	inf	10.0
null	null	F	B	null	D

Path from B to A: does not exist.
Path from B to C: B -> D -> F -> C
Path from B to D: B -> D
Path from B to E: does not exist.
Path from B to F: B -> D -> F

Graph #4 - Undirected Graph

{A=[B22.0, D15.0, E3.0], B=[A22.0, D4.0, C1.0], C=[B1.0, D25.0, F8.0],
D=[A15.0, B4.0, F6.0, C25.0, E2.0], E=[A3.0, D2.0], F=[D6.0, C8.0]}

Minimum Distances:

A	B	C	D	E	F
3.0	inf	inf	2.0	0.0	inf
E	null	null	E	null	null

Minimum Distances:

A	B	C	D	E	F
3.0	6.0	27.0	2.0	0.0	8.0
E	D	D	E	null	D

Minimum Distances:

A	B	C	D	E	F
3.0	6.0	27.0	2.0	0.0	8.0
E	D	D	E	null	D

Minimum Distances:

A	B	C	D	E	F
3.0	6.0	7.0	2.0	0.0	8.0
E	D	B	E	null	D

Minimum Distances:

A	B	C	D	E	F
3.0	6.0	7.0	2.0	0.0	8.0
E	D	B	E	null	D

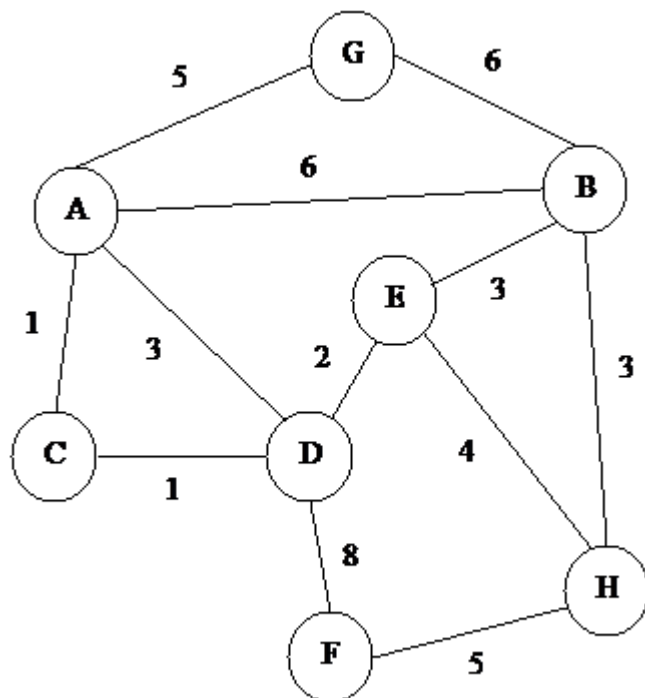
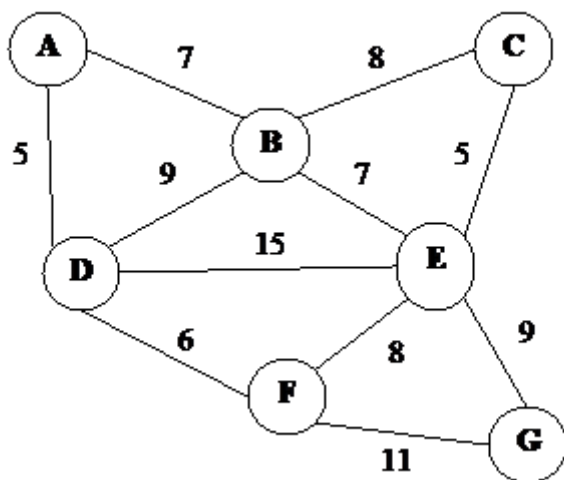
Minimum Distances:

A	B	C	D	E	F
---	---	---	---	---	---

3.0	6.0	7.0	2.0	0.0	8.0
E	D	B	E	null	D

Path from E to A: E -> A
 Path from E to B: E -> D -> B
 Path from E to C: E -> D -> B -> C
 Path from E to D: E -> D
 Path from E to F: E -> D -> F

Sample Undirected Graphs :



Sample Output :

Graph five:

Path from A to B: A -> B
 Path from A to C: A -> B -> C
 Path from A to D: A -> D
 Path from A to E: A -> B -> E
 Path from A to F: A -> D -> F
 Path from A to G: A -> D -> F -> G

Graph six:

Path from A to B: A -> B
 Path from A to C: A -> C
 Path from A to D: A -> C -> D
 Path from A to E: A -> C -> D -> E
 Path from A to F: A -> C -> D -> F
 Path from A to G: A -> G
 Path from A to H: A -> C -> D -> E -> H

The costs / previous node updates are shown below on the following pages:

Graph #5 - Undirected Graph

```
{A=[B7.0, D5.0], B=[A7.0, C8.0, D9.0, E7.0], C=[B8.0, E5.0],
D=[A5.0, B9.0, E15.0, F6.0], E=[B7.0, C5.0, D15.0, F8.0, G9.0],
F=[D6.0, E8.0, G11.0], G=[E9.0, F11.0]}
```

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	inf	5.0	inf	inf	inf
null	A	null	A	null	null	null

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	inf	5.0	20.0	11.0	inf
null	A	null	A	D	D	null

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	15.0	5.0	14.0	11.0	inf
null	A	B	A	B	D	null

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	15.0	5.0	14.0	11.0	22.0
null	A	B	A	B	D	F

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	15.0	5.0	14.0	11.0	22.0
null	A	B	A	B	D	F

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	15.0	5.0	14.0	11.0	22.0
null	A	B	A	B	D	F

Minimum Distances:

A	B	C	D	E	F	G
0.0	7.0	15.0	5.0	14.0	11.0	22.0
null	A	B	A	B	D	F

```
Path from A to B: A -> B
Path from A to C: A -> B -> C
Path from A to D: A -> D
Path from A to E: A -> B -> E
Path from A to F: A -> D -> F
Path from A to G: A -> D -> F -> G
```

Graph #6 - Undirected Graph

```
{A=[B6.0, C1.0, D3.0, G5.0], B=[A6.0, E3.0, G6.0, H3.0],
C=[A1.0, D1.0], D=[A3.0, C1.0, E2.0, F8.0], E=[B3.0, D2.0, H4.0],
F=[D8.0, H5.0], G=[A5.0, B6.0], H=[B3.0, E4.0, F5.0]}
```

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	3.0	inf	inf	5.0	inf
null	A	A	A	null	null	A	null

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	inf	inf	5.0	inf
null	A	A	C	null	null	A	null

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	inf
null	A	A	C	D	D	A	null

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	8.0
null	A	A	C	D	D	A	E

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	8.0
null	A	A	C	D	D	A	E

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	8.0
null	A	A	C	D	D	A	E

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	8.0
null	A	A	C	D	D	A	E

Minimum Distances:

A	B	C	D	E	F	G	H
0.0	6.0	1.0	2.0	4.0	10.0	5.0	8.0
null	A	A	C	D	D	A	E

Path from A to B: A -> B

Path from A to C: A -> C

Path from A to D: A -> C -> D

Path from A to E: A -> C -> D -> E

Path from A to F: A -> C -> D -> F

Path from A to G: A -> G

Path from A to H: A -> C -> D -> E -> H

If you need more help for graph five!

	A	B	C	D	E	F	G
A	0,	7,	0,	5,	0,	0,	0
B	7,	0,	8,	9,	7,	0,	0
C	0,	8,	0,	0,	5,	0,	0
D	5,	9,	0,	0,	15,	6,	0
E	0,	7,	5,	15,	0,	8,	9
F	0,	0,	0,	6,	8,	0,	11
G	0,	0,	0,	0,	9,	11,	0

```
prev[ 0, 0, 0, 0, 0, 0, 0] // set up
vis [ f, f, f, f, f, f, f]
cost[ 0, ∞, ∞, ∞, ∞, ∞, ∞]
```

1. Go thru cost and find the minimum value from nodes not visited. The min cost is 0 at index 0 (node A) and mark it visited.

2. Now go thru the matrix of A's row and check if the distance from A to each node plus the min cost is less than the value in the corresponding cost array. Only check the one's that haven't been visited yet as well as the one's that have a path. Update the cost and prev if applicable.

```
prev[ 0, 0, 0, 0, 0, 0, 0] // first pass
vis [ t, f, f, f, f, f, f]
cost[ 0, 7, ∞, 5, ∞, ∞, ∞] // after updating
```

3. repeat steps 1 and 2

1. Go thru cost and find the minimum value from nodes not visited. The min cost is 5 at index 3 (node D) and mark it visited.

2. Now go thru the matrix of D's row and check if the distance from D to each node plus the min cost is less than the value in the corresponding cost array. Only check the one's that haven't been visited yet as well as the one's that have a path. Update the cost and prev if applicable.

```
prev[ 0, 0, 0, 0, 3, 3, 0] // second pass
vis [ t, f, f, t, f, f, f]
cost[ 0, 7, ∞, 5, 20, 11, ∞] // after updating
```

4. repeat steps 1 and 2

1. min cost of unvisited nodes is 7 at index 1 (node B) - mark it now visited

2. go thru B's row of the matrix and check if the cost to B plus the distance from B is less than the corresponding value in the cost array. Only check the one's that haven't been visited yet as well as the one's that have a path. Update the cost and prev if applicable.

```
prev[ 0, 0, 1, 0, 1, 3, 0] // third pass
vis [ t, t, f, t, f, f, f]
cost[ 0, 7, 15, 5, 14, 11, ∞] // after updating
```

5. repeat steps 1 and 2.

1. min cost of unvisited nodes is 11 at index 5 (node F) - mark it now visited

2. go thru F's row of the matrix and check if the cost to F plus the distance from F is less than the corresponding value in the cost array. Only check the one's that haven't been visited yet as well as the one's that have a path. Update the cost and prev if applicable.

```
prev[ 0,    0,    1,    0,    1,    3,    5]    // third pass
vis [ t,    t,    f,    t,    f,    t,    f]
cost[ 0,    7,   15,    5,   14,   11,   22]    // after updating
```

6. repeat this process a total of n times